Routh Array and Root Locus Design

R0005E - Measurement and Feedback Control

Wolfgang Birk

Department of Computer Science, Electrical and Space Engineering

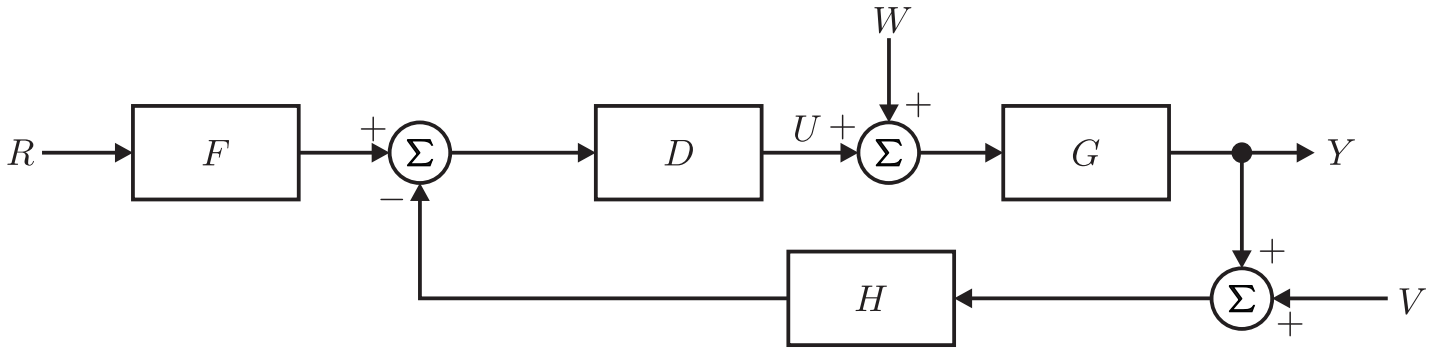
LULEÅ
UNIVERSITY
OF TECHNOLOGY

Roots of a closed loop system

We have already seen that the poles of a closed loop control system depend on the controller parameters. In some cases, we can place the poles freely, which means that there are sufficiently many parameters in the controller that can be adjusted to achieve any desired location of the poles in the complex s-plane.

If this is not possible, then the roots of the characteristic polynomial will attain certain locations in the complex s-plane, depending on the controller parameters.

Assume we have a general closed loop system



In this setup, the characteristic polynomial is given by

$$P(s) = 1 + H(s)G(s)D(s) \tag{1}$$

Stability of a closed loop system

We know that the stability of the closed loop system requires that the roots of the characteristic polynomial have to resides in the left half plane. Instead of solving the characteristic polynomial for the roots, there is a necessary and sufficient condition for stability, the so-called **Routh's stability criterion**.

Procedure:

1. State the characteristic polynomial in the form

$$s^n + a_1 s^{n-1} + a_s s^{n-2} + \quad \cdots \quad + a_{n-1} s + a_n$$

2. Setting up the Routh Array, which has $(n+1)$ rows.

$$\begin{array}{r|llll} s^n & 1 & a_2 & a_4 & \cdots \\ s^{n-1} & a_1 & a_3 & a_5 & \cdots \\ s^{n-2} & b_1 & b_2 & b_3 & \cdots \\ s^{n-3} & c_1 & c_2 & c_3 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \end{array}$$

3. Calculate b_*

$$b_1 = -\frac{\det \begin{bmatrix} 1 & a_2 \\ a_1 & a_3 \end{bmatrix}}{a_1}, \qquad b_2 = -\frac{\det \begin{bmatrix} 1 & a_4 \\ a_1 & a_5 \end{bmatrix}}{a_1}, \qquad b_3 = -\frac{\det \begin{bmatrix} 1 & a_6 \\ a_1 & a_7 \end{bmatrix}}{a_1}, \cdots$$

Stability of a closed loop system (2)

4. Calculate c_*

$$c_1 = -\frac{\det\begin{bmatrix} a_1 & a_3 \\ b_1 & b_2 \end{bmatrix}}{b_1}, \qquad c_2 = -\frac{\det\begin{bmatrix} a_1 & a_5 \\ b_1 & b_3 \end{bmatrix}}{b_1}, \qquad c_3 = -\frac{\det\begin{bmatrix} a_1 & a_7 \\ b_1 & b_4 \end{bmatrix}}{b_1}, \cdots$$

5. Check the condition: *The system is stable if and only if all elements in the first column of the Routh Array are positive*.

6. Determine the number of RHP poles by counting the number of sign changes in the first column.

An important aspect about the Routh Array is that it can be calculated with the controller parameters in array. Thereby, conditions for the controller parameters can be found which yield a stable closed loop system. These conditions can be used to determine stability regions.

Root Locus of a closed loop system

Let's assume all the transfer functions $H(s)$, $G(s)$ and $D(s)$ can be represented in the ZPK representation, then the controller will have a gain gain constant K.

$$D(s) = K \cdot D'(s)$$

Now we can rewrite (1) into the form:

$$P(s) = 1 + KL(s), \quad \text{with} \quad L(s) = H(s)G(s)D'(s)$$

$L(s)$ is also the open loop transfer function and can be written as

$$L(s) = \frac{b(s)}{a(s)} = \frac{\prod_{i=1}^{m}(s - z_i)}{\prod_{j=1}^{n}(s - p_j)}$$

The roots of $P(s)$ will then satisfy the following equation:

$$L(s) = -\frac{1}{K} \tag{2}$$

Thus, for a given $K \geq 0$, there will be a set of roots that fulfill this equation. The dynamic behavior of the closed loop system can then be inferred from the pole locations for a changing K.

Root locus in Matlab:

- `rlocus`: Determine the root locus for $L(s)$ and returns the root locations for different K.

Root Locus of a closed loop system (2)

- `rlocusplot`: Produces a plot for `rlocus`
- `rltool`: Design tool for controllers based on the root locus

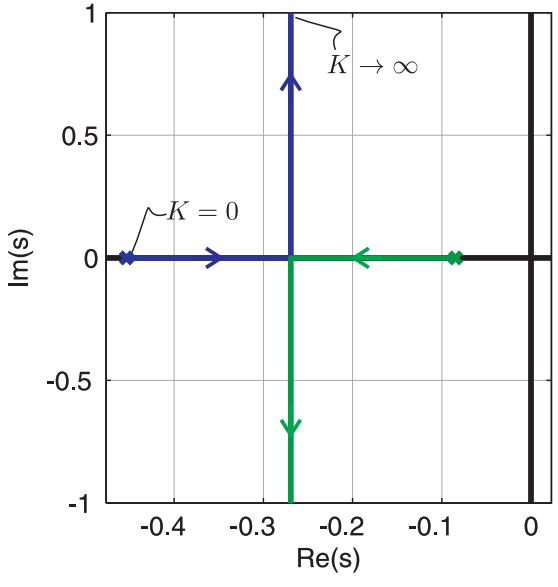
Example of two root locus plots

We will now revisit two processes from before:

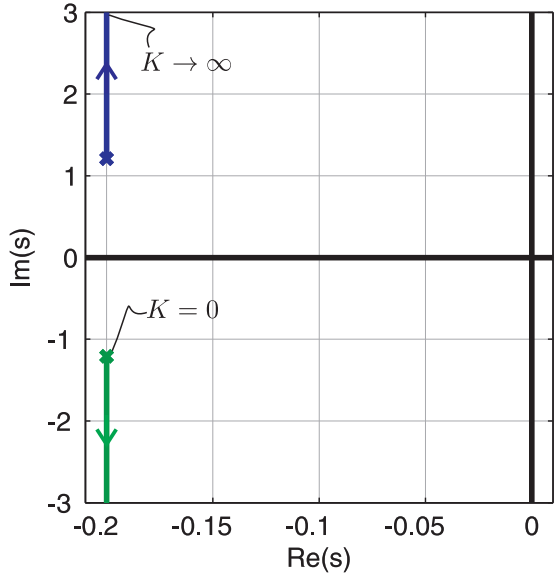
- Example from the tuning rules: $G_1(s) = \dfrac{1}{26s^2 + 14s + 1} = \dfrac{0.15}{(s + 0.45)(s + 0.08)}$
- Traverse crane: $G_2(s) = \dfrac{1.5}{s^2 + 0.4s + 1.5} = \dfrac{1.5}{(s + 0.2 + j1.21)(s + 0.2 - j1.21)}$

If we assume $D'(s) = 1$, then the following root locus plots can be found:

$G_1(s)$



$G_2(s)$



Example of two root locus plots (2)

Observations:

- For $K = 0$, the root locations coincide with the pole locations of the open loop transfer function.
- When K is increased until infinity, then the root location tend to infinity, too.
- There are as many branches as there are poles.
- The roots in the plot are the poles for the closed loop system with a P controller having the gain K.

Note:

The root locus plot will not be constructed by hand and just because there are tools now that can recalculate the locus plot on the fly, the method has become popular again.

Construction of the root locus

Although, the drawing will not be done by hand, some of the rules for the construction are helpful, when designing a controller using the root locus.

Some construction rules:

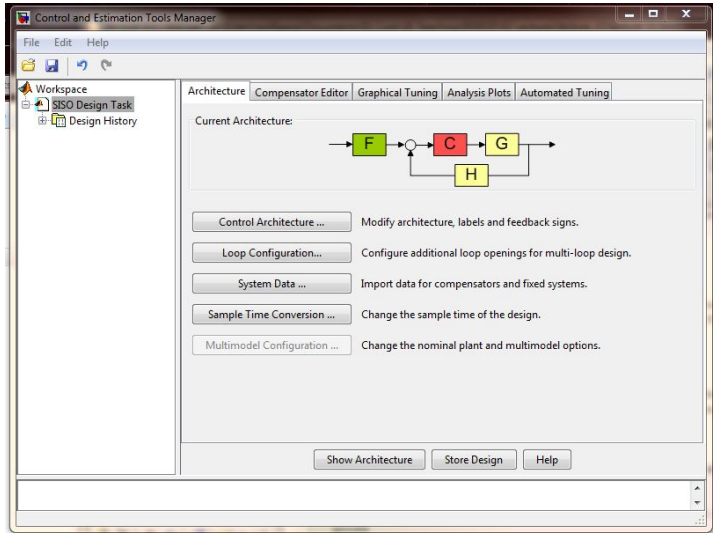
1. The n branches will start at the poles of $L(s)$ and m branches will end at the zeros of $L(s)$.
2. The loci are on the real axis to the left of an odd number of poles an zeros.
3. For large s and K, $n-m$ of the loci are asymptotic to lines at angles ϕ_l radiating out from the center point $s=\alpha$ on the real axis:

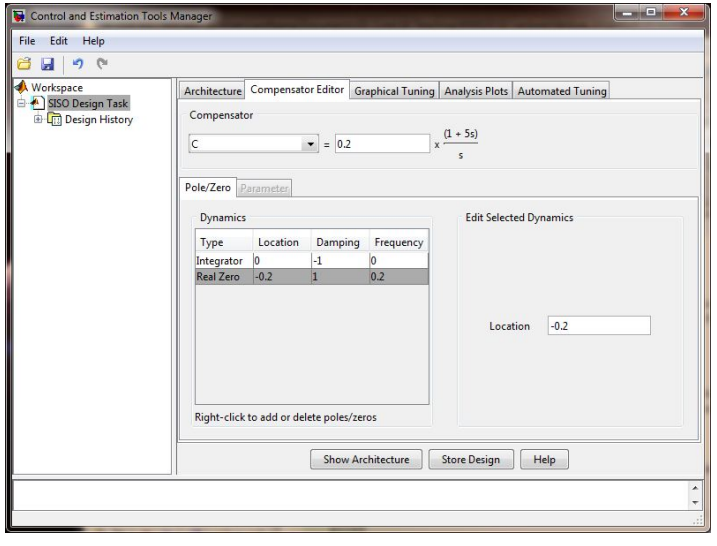
$$\phi_l = \frac{180^\circ + 360^\circ(l-1)}{n-m}, \qquad l = 1 \ldots n-m$$
$$\alpha = \frac{\sum p_j - \sum z_i}{n-m}$$

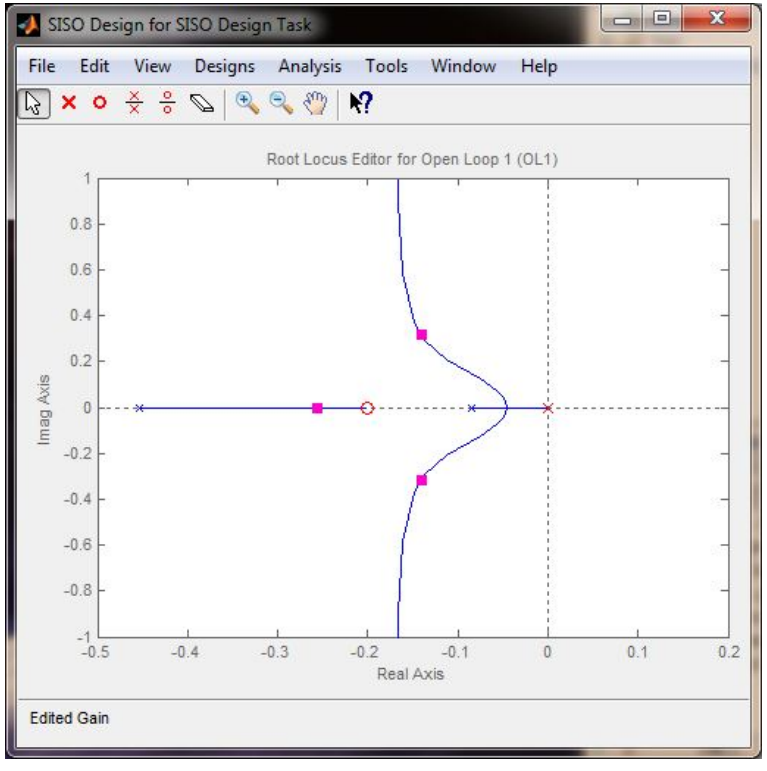
Matlab: `rltool`

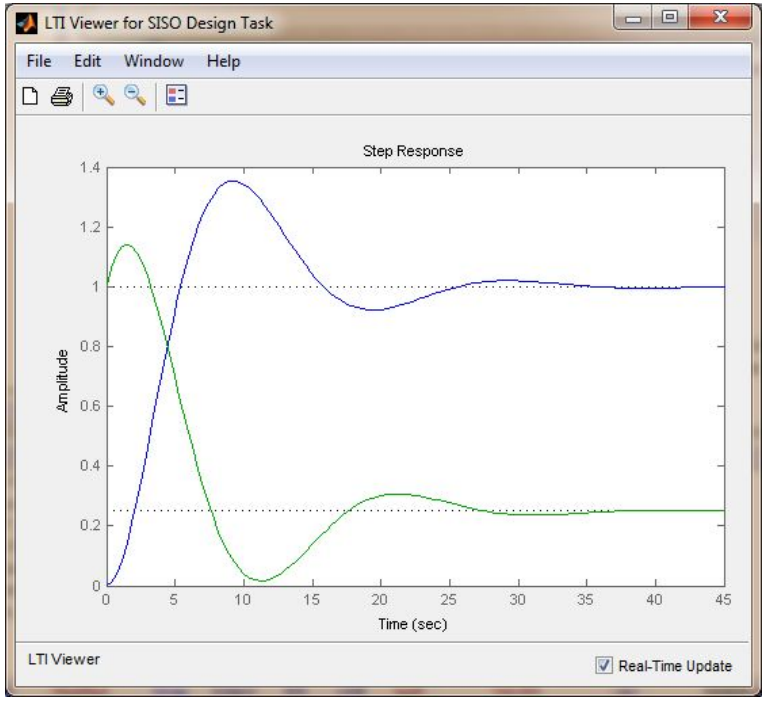
In Matlab you can use rltool to re-calculate the root locus in real time. The tool also updates analysis plots of the closed loop system on the fly:

- Root locus is updated as soon as you change the number/placement of poles/zeros of the compensator (controller).
- Draging of roots results in automatic adjustment of the compensator gain.
- Simulation of selected analysis plot of the closed loop upon changes in the compensator.
- Optimization of a compensator for a specification.
- Discretization of a compensator.









- Left: Root locus update with current closed loop poles (square in magenta), user defined poles and zeros (crosses, circles in red), system poles and zeros (crosses, circles in blue).
- Right: Analysis plot of the step response for the transfer functions from $r \to y$ (blue) and $r \to u$ (green).

Controller design

Depending on the controller, there are additional open loop poles and zeros ($K = 0$) that have to be selected.

P - Controller:

This is only a gain. Just use the root locus setup as it is.

PI - Controller:

$$D(s) = k_P + \frac{k_I}{s} = k_P \frac{s + \frac{1}{T_I}}{s}$$

- Poles: $s = 0$
- Zeroes: $s = -\frac{1}{T_I}$
- Conclusion: You have to place a pole at origo and a zero for a pre-selected T_I. The T_I will affect the shape of the root locus.

PD - Controller:

$$D(s) = k_P + k_D s = k_P T_D (s + \frac{1}{T_D})$$

- Poles: none
- Zeroes: $s = -\frac{1}{T_D}$
- Conclusion: You have to place a zero for a pre-selected T_D. Again, the T_D will affect the shape of the root locus.

Controller design (2)

PID - Controller:

$$D(s) = k_P + \frac{k_I}{s} + k_D s = k_P T_D \frac{s^2 + \frac{1}{T_D}s + \frac{1}{T_I T_D}}{s}$$

We can now determine the zeros depending on the parameters T_D and T_I:

$$D(s) = k_P T_D \frac{\left(s + \frac{1}{2T_D} + \sqrt{\frac{1}{4T_D^2} - \frac{1}{T_I T_D}}\right)\left(s + \frac{1}{2T_D} - \sqrt{\frac{1}{4T_D^2} - \frac{1}{T_I T_D}}\right)}{s}$$

- Poles: $s = 0$
- Zeroes: $s = -\frac{1}{2T_D} \pm \sqrt{\frac{1}{4T_D^2} - \frac{1}{T_I T_D}}$
- Conclusion: You have to place a pole at origo and two zeroes for a pre-selected T_D and T_I. Here, you could choose to create complex zeroes.

Other compensators:

There are other types of compensators. Those usually have an affect on the process to increase stability, remove oscillations or attain somewhat better regulation properties.

- Lead compensator $D(s) = K\frac{s+z}{s+p}$, with $z < p$ reduces oscillations and increase stability.
- Lag compensator $D(s) = K\frac{s+z}{s+p}$, with $z > p$ improves regulation properties.
- Notch compensators can attenuate oscillations. They are tricky to design in root locus!

Example G_1

We have seen that a P Controller will introduce oscillations or a large steady state control error.

Specification:

- Small overshoot, less than 10%
- No control error
- Faster response time

We will try to design a PI Controller:

$$G_1(s) = \frac{0.15}{(s+0.45)(s+0.08)}, \qquad D(s) = k_P \frac{s + \frac{1}{T_I}}{s}$$

Some observations:

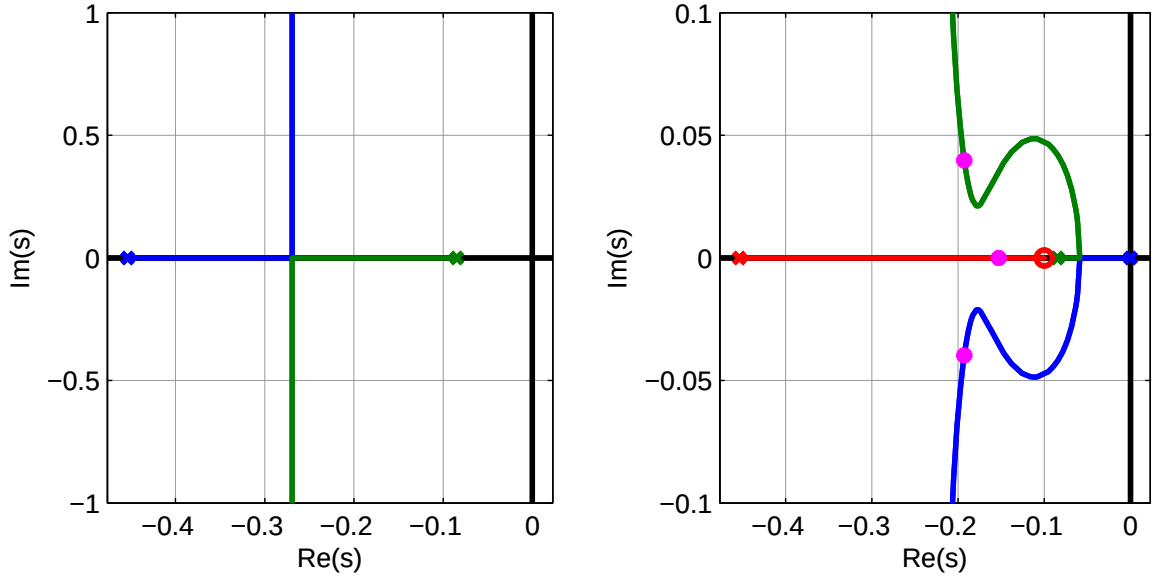
- There are three poles, thus three branches.
- The additional zero will capture one of the branches and will be the end point for one branch.
- We have to find a placement of the zero so that the process would respond equally fast with little overshoot.

Suggested PI controller:

$$D(s) = 0.385 + \frac{0.0385}{s}$$

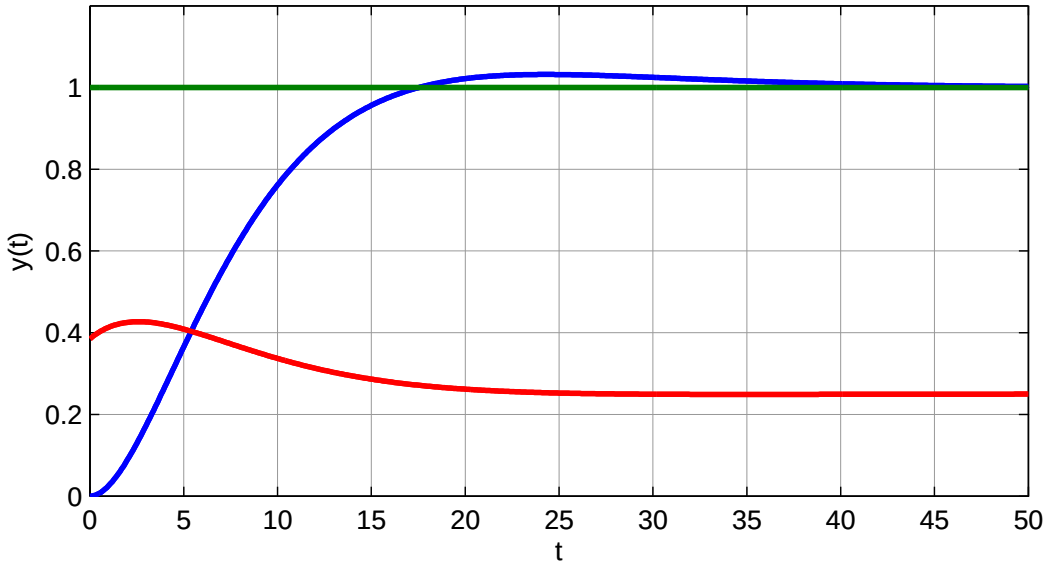
This controller has the parameters: $k_P = 0.0385$ and $T_I = 10$.

Root locus with P and PI controller:



Example G_1 (3)

Step response: r (green), $r \to y$ (blue), $r \to u$ (red)



Conclusion: The closed loop system fulfills the specification.

Example G_2

We have seen that a P Controller will always introduce oscillations and create a large steady state control error.

Specification:

- Small overshoot, less than 10%
- No control error
- Faster response time

Again, we will try to design a PI Controller:

$$G_2(s) = \frac{1.5}{s^2 + 0.4s + 1.5} = \frac{1.5}{(s + 0.2 + j1.21)(s + 0.2 - j1.21)}, \qquad D(s) = k_P \frac{s + \frac{1}{T_I}}{s}$$

Some observations:

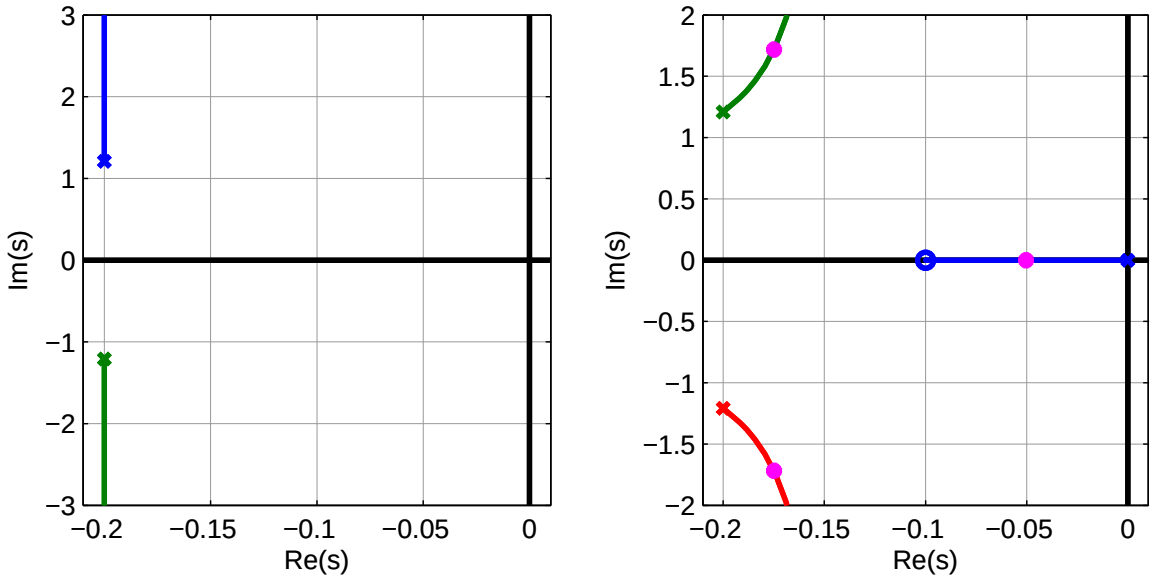
- There are three poles, thus three branches.
- The additional zero will capture one of the branches and will be the end point for one branch.
- We have to find a placement of the zero so that the process would respond equally fast with little overshoot.

Tried PI controller:

$$D(s) = 1 + \frac{0.1}{s}$$

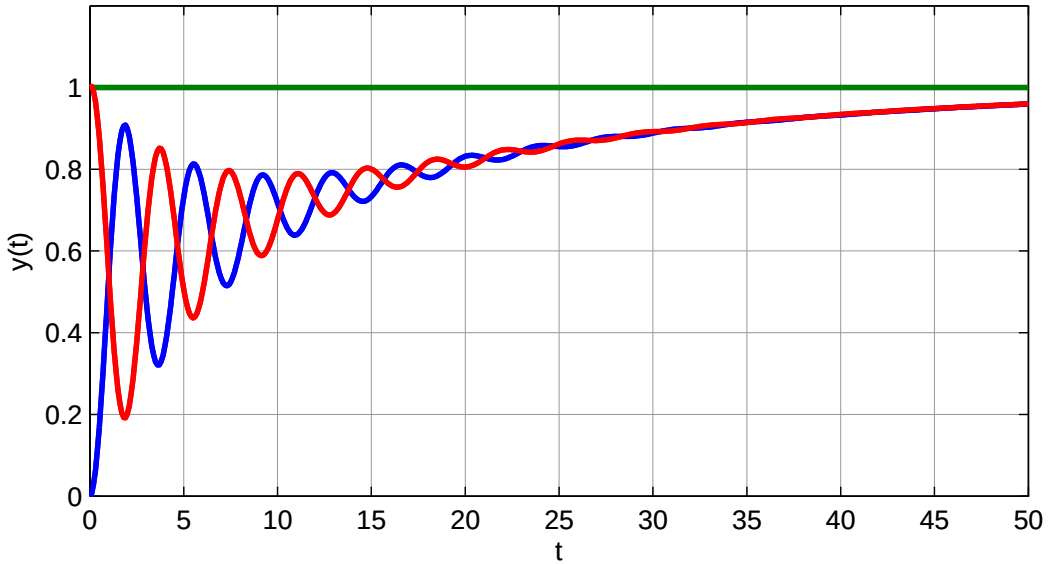
This controller has the parameters: $k_P = 1$ and $T_I = 10$.

Root locus with P and PI controller:



Example G_2 (3)

Step response: r (green), $r \to y$ (blue), $r \to u$ (red)



Conclusion:

- The closed loop system is both slow and oscillative.
- The control does diminish.
- The closed loop system will always be slower.

Example G_2 (4)

Now we will try to design a PID Controller:

$$G_2(s) = \frac{1.5}{s^2 + 0.4s + 1.5} = \frac{1.5}{(s + 0.2 + j1.21)(s + 0.2 - j1.21)}, \qquad D(s) = k_P \frac{s + \frac{1}{T_I} + T_D s}{s}$$

Some observations:

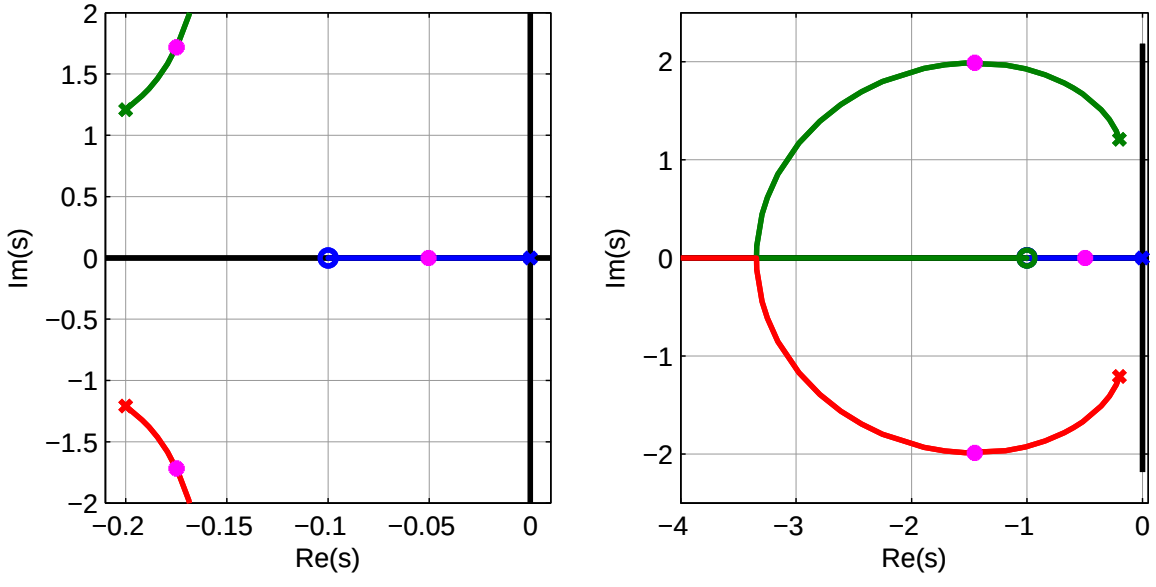
- There are three poles, thus three branches.
- There are two zeros that will capture two of the branches and will be the end point for two branches.
- We have to find a placement for two zeroes that would move the branches more to the left.

Suggested PID controller:

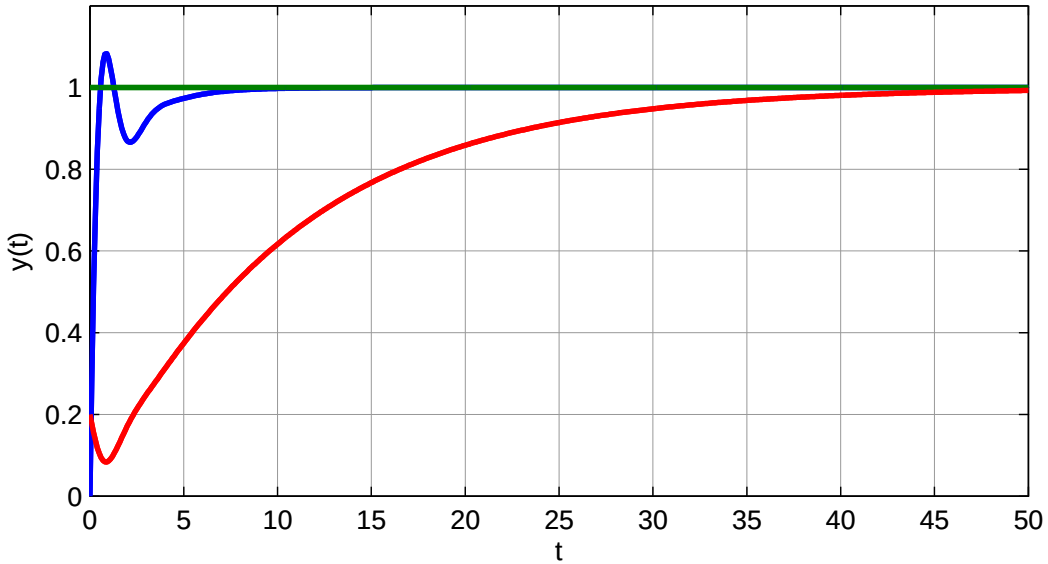
$$D(s) = 4 + \frac{2}{s} + 2s$$

This controller has the parameters: $k_P = 4$, $T_I = 0.5$ and $T_D = 0.5$.

Root locus with PI and PID controller:



Step response: r (green), $r \to y$ (blue), $r \to u$ (red)



Conclusion:

- The closed loop system is responding fast and a slight overshoot.
- The control error does diminish.
- The closed loop system is responding faster than the original system.

Additional material

Routh-Hurwitz criterion

- Routh-Hurwitz Criterion, An Introduction
  `https://www.youtube.com/watch?v=WBCZBOB3LCA&t=2s`
- Routh-Hurwitz Criterion, Special Cases
  `https://www.youtube.com/watch?v=oMmUPvn6lP8`
- Routh-Hurwitz Criterion, Beyond Stability
  `https://www.youtube.com/watch?v=wGC5C_7Yy-s`

Root Locus method

- The Root Locus Method - Introduction
  `https://www.youtube.com/watch?v=CRvVDoQJjYI`
- Gain a better understanding of Root Locus Plots using Matlab
  `https://www.youtube.com/watch?v=pG3_b7wuweQ`
- Root Locus Plot: Common Questions and Answers
  `https://www.youtube.com/watch?v=WLBszzT0jp4`

There are more videos on Root Locus, which you can view if you have a deeper interest in the method and how to design controllers (also called compensators)